南京郵電大學
Nanjing University of Posts and Telecommunications

# Python程序设计（混合式）

py for AI

张伯雷
南京邮电大学计算机学院，基础教学中心

https://bolei-zhang.github.io/course/python-ai.html

2026/3/16

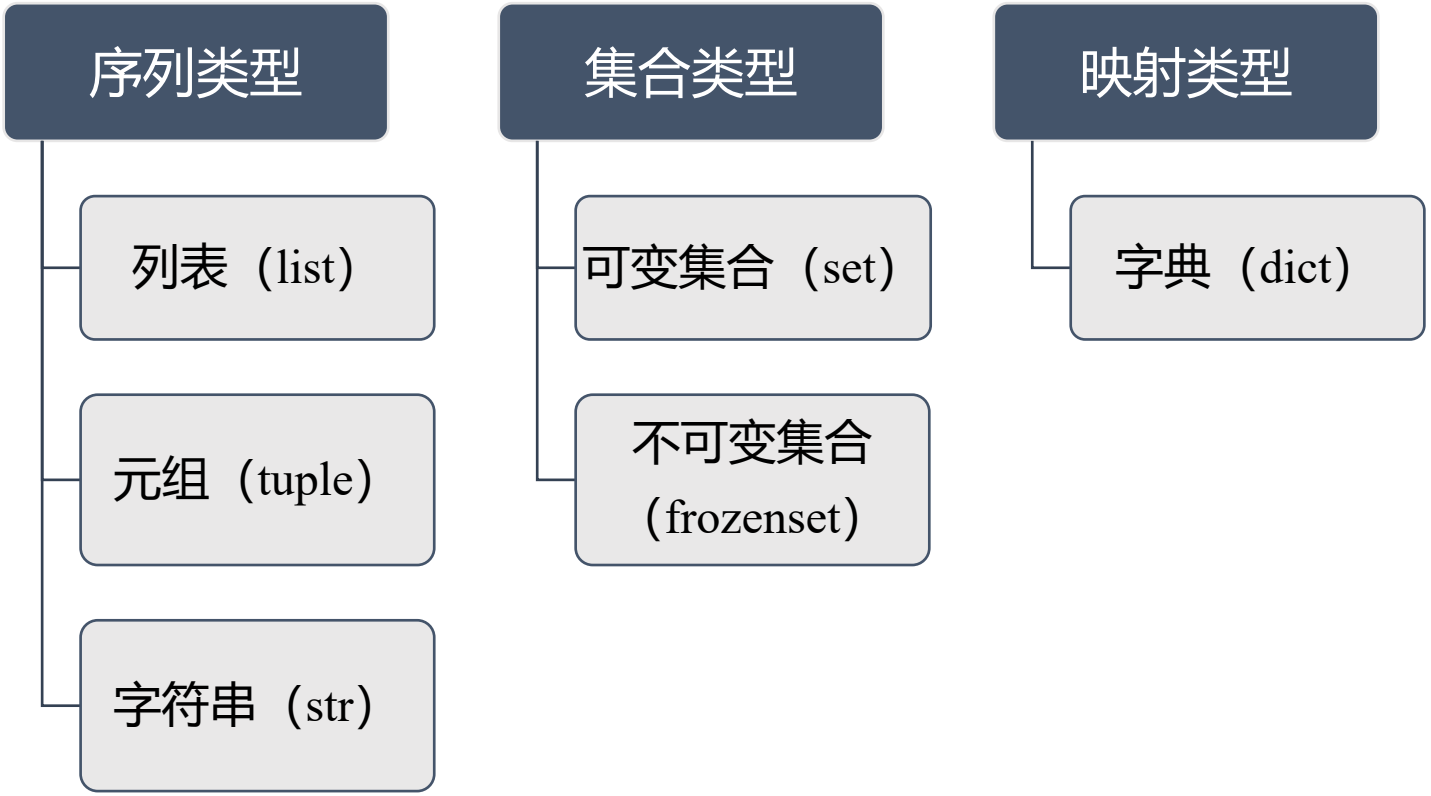
# 目录




- Python基本语法
  - 变量和数据类型
  - 运算符和表达式
  - 字符串
  - 流程控制
- Python组合数据类型
- Python函数
- 异常处理和文件操作
- Python面向对象基础
- 数值计算库NumPy

# 组合数据类型

- 组合数据类型可以将多个数据组织起来，根据数据组织方式的不同，Python的组合数据类型可分成三类：**序列**类型、**集合**类型和**映射**类型



# 组合数据类型——列表

- 列表（list）是一种**序列数据类型**，是一种**可变**数据类型，内部元素可以任意增删和修改
- 列表对象的**定义和基本运算**如下：

```
#代码2.16 列表对象的定义和基本运算
lst1 = []  #定义一个空列表
#len()函数可以返回参数对象中包含元素的数量
print(f"len(lst1) = {len(lst1)}")
#list()函数可以将参数对象转换成列表
lst1 = list("Python!")
#列表也是序列类型对象，所以支持索引和切片
print(f"lst1[2] = {lst1[2]}, lst1[-2] = {lst1[-2]}")
print(f"lst1[2:-2] = {lst1[2:-2]}")
```

# 组合数据类型——列表

- 列表（list）是一种**序列数据类型**，是一种**可变**数据类型，内部元素可以任意增删和修改
- 列表对象的**定义和基本运算**如下：

```
lst1 = [1,2,3]
lst2 = [1,2,3]
#通过id()函数我们可以取得对象的身份标识
print(f"{id(lst1)=}, {id(lst2)=}")
print(“lst1和lst2是否相等：”，lst1==lst2) print(“lst1和lst2是否相同：”, lst1 is lst2)
#不同的变量与同一个列表对象关联
lst1 = lst2 = [1,2,3]
#无论操作变量lst1或者lst2，其实都是在操作同一个列表
lst1[0] = 100
```

# is/==

- Python 中对象包含的三个基本要素，分别是：id(身份标识)、type(数据类型)和value(值)

| 要素 | 说明 |
| --- | --- |
| id | 身份标识，也是内存地址，用来唯一标识一个对象 |
| type | 数据类型 |
| value | 值 |

- ==
  - 比较操作符，用来比较判断两个对象的 **value(值)** 是否相等
- is
  - 同一性运算符，判断的是对象间的 **唯一身份标识**，也就是 **id** 是否相同

# 列表的常用方法



- 列表对象具有一系列定义好的方法可以直接使用

```
#代码2.17 列表对象的常用方法
lst1 = list("Python")
lst1.append("666")   #追加元素到列表中
lst1.insert(1, "is")   #插入元素到列表中
lst1.remove("i")   #从列表中移除指定的元素

#从指定位置弹出列表元素，默认弹出列表的最后一个元素
print(f'lst1.pop(0) = {lst1.pop(0)} ')

#将参数转换成列表元素后，连接到原列表的末尾
lst1.extend([1, 2, 3])
```

# 列表的常用方法



- 列表对象具有一系列定义好的方法可以直接使用

```
#对列表元素按照ASCII码从小到大的顺序进行排序
lst1.sort()
#在列表中对指定的内容进行计数
print("'o'在lst1中出现了", lst1.count("o"), "次")
#在列表中查找指定的内容，返回其索引值
print("'t'在lst1中的索引值是：", lst1.index("t"))
lst1 = [1,2,3]
#通过copy()方法可以复制原列表对象，得到一个新的列表对象(查看id是否相同的地址)
lst2 = lst1.copy()
lst2[0] = 100
print(f"{lst1 = }, {lst2 = }")
```

# Python解释器

```
>>> L = []
>>> dir(L)
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__',
'__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__',
'__imul__', '__init__', '__iter__', '__le__', '__len__', '__lt__',
'__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',
'__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__',
'__sizeof__', '__str__', '__subclasshook__', 'append', 'clear',
'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove',
'reverse', 'sort']
>>> [d for d in dir(L) if '__' not in d]
['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 're

>>> help(L.append)
Help on built-in function append:

append(...)
    L.append(object) -> None -- append object to end

>>> L.append(1)
>>> L
[1]
```

dir() 函数用于获取一个对象的所有属性和方法的列表。它返回一个包含对象的属性（包括方法）的字符串列表。这个列表通常用于了解对象的可用功能，帮助你在编程中探索和使用它。

# 列表推导式



- 列表推导式将**列表元素的生成规则放置在一对方括号内完成列表的创建**

```
#代码2.18 列表推导式的使用
lst1 = [n for n in range(1,10) if n%2==1]   #创建列表包含10以内的奇数

lst2 = [5*n for n in range(5)]    #创建等差数列构成的列表

lst3 = [3**n for n in range(5)]   #创建等比数列构成的列表

#判断元素是否在数组里
5 in lst2
```

# 思考



- Python列表是数组还是链表?

## 列表是如何在CPython中实现的?

CPython的列表实际上是可变长度的数组，而不是lisp风格的链表。该实现使用对其他对象的引用的连续数组，并在列表头结构中保留指向该数组和数组长度的指针。

这使得索引列表 `a[i]` 的操作成本与列表的大小或索引的值无关。

当添加或插入项时，将调整引用数组的大小。并采用了一些巧妙的方法来提高重复添加项的性能；当数组必须增长时，会分配一些额外的空间，以便在接下来的几次中不需要实际调整大小。

# 基于列表的栈和队列

- 栈
  - ✓ 先进后出，后进先出
  - ✓ 入栈：append()
  - ✓ 出栈：pop()

- 队列
  - ✓ 先进先出
  - ✓ 入队：append()
  - ✓ 出队：pop(0)

# append和insert，哪个效率更高?

```
l = []
for i in range(int(1e7)):
    l.append(i)
```

```
l = []
for i in range(int(1e7)):
    l.insert(0, i)
```

pop和pop(0)呢，请大家思考?

# 组合数据类型——元组



- 元组（tuple）也是一种序列数据类型，但是它是一种**不可变数据**类型，其**元素不可增删改**

```
#代码2.19 元组对象的定义和基本运算
#定义空元组、一元组、二元组
tup1,tup2,tup3 = tuple(),(1,),(2,3)
#通过+运算连接两个旧元组，得到一个新元组
tup1 = tup2 + tup3
tup1 = tuple("Python!")
#元组也支持索引和切片运算
print(f"{tup1[2]}, {tup1[3:6]}")
#由于元组是不可变对象，元素不可以修改，所以该语句报错
tup1[2] = 'a'
```

# 组合数据类型——元组



- 元组（tuple）也是一种序列数据类型，但是它是一种**不可变数据**类型，其**元素不可增删改**

```
#代码2.20 元组对象的常用方法
#tuple()函数可以将参数对象转换成元组
tup1 = tuple("Beautiful is better than ugly.")

#统计指定的参数在元组中出现的次数
print(f" tup1.count('t') = {tup1.count('t') }")

#查找指定的参数，并返回其在元组中的索引值
print(f" tup1.index('a') = {tup1.index('a')}")
```

# 列表与元组的区别

- 性能
  - ✓ 由于元组是**不可变的**，它们通常比列表具有更高的性能，因为不需要考虑元素的添加或删除
  - ✓ 列表的**可变性**可能导致更多的内存分配和复制操作，从而在某些情况下影响性能。
- 应用场景
  - ✓ 列表通常用于需要动态增长和修改元素的情况，例如存储一组项目或记录
  - ✓ 元组通常用于表示不可更改的数据集，例如坐标、日期时间、函数的多返回值等

# 组合数据类型——字典

- 字典（dict）是表示**映射关系的类型**，由**键-值**对构成字典中的元素，是一种**可变**数据类型
- 字典不是序列类型，**其中元素不能使用索引值进行访问，而是通过键访问对应元素，键必须唯一，不可变**

```
#代码2.21 字典对象的定义和基本运算
dct1 = {}   #定义一个空字典

#字典元素的键可以是整数、字符串和元组等不可变对象
dct1 = {1:"Python", "Tom":"Male", (3,4):"Red"}

dct1[(3,4)] = "Blue"   #字典的元素是可以被修改的
del dct1[1] #字典的元素可以被删除
dct1[2] = "Java"   #字典中可以随时添加元素
print(f"After change：{dct1}")
```

# 字典的常用方法

```
#keys()方法可以返回字典中的所有键
print(f"{dct1.keys() }")
#values()方法可以返回字典中的所有值
print(f"{dct1.values() }")
#items()方法可以返回字典中的所有键-值对
print(f"{dct1.items() }")
#get()方法用来返回以参数1作为键的元素值，如果找不到则返回参数2的内容
print(f"{dct1.get('Tom','Unknown') }")
print(f"{dct1.get('Jerry','Unknown') }")
#copy()方法通过拷贝原字典数据创建一个新的字典对象
dct2 = dct1.copy()
print(f"{dct2}")
```

# 字典的常用方法

```
#popitem()方法用于弹出最后被加入字典的元素
print(f"{dct2.popitem()}")
#pop()方法用于弹出字典中的指定元素，并返回其值
print(f"{dct2.pop('Tom')}")
#update() 作用是使用参数对象的内容对字典进行更新
dct2.update({'Jerry':'Male})
#clear()方法可以将字典中的元素清空
print(f"{dct2.clear()}")
#fromkeys()用参数1作为键，参数2作为值，创建字典
dct3 = dict.fromkeys(["Alice","Mary"],"Female")
#setdefault()用于以参数作为键-值对插入元素
print(f"{dct3.setdefault('Alice','Male')}")
```

# 字典是如何实现的?



## 字典是如何在CPython中实现的?

CPython的字典实现为可调整大小的哈希表。与B-树相比，这在大多数情况下为查找（目前最常见的操作）提供了更好的性能，并且实现更简单。

字典的工作方式是使用 `hash()` 内置函数计算字典中存储的每个键的hash代码。hash代码根据键和每个进程的种子而变化很大；例如，"Python" 的hash值为-539294296，而"python"(一个按位不同的字符串)的hash值为1142331976。然后，hash代码用于计算内部数组中将存储该值的位置。假设您存储的键都具有不同的hash值，这意味着字典需要恒定的时间 -- O(1)，用Big-O表示法 -- 来检索一个键。

# 使用list，dict还是tuple?

- 场景：学生管理系统，每个学生有学号、姓名、班级、成绩等信息

- 功能1：对于学生成绩进行排序

```
self.students.sort(key=lambda x: x.score)
```

- 功能2：根据学号修改成绩等信息

```
self.students[student_id]['score'] = new_score
```

# 组合数据类型——集合



- 集合是一种非序列容器对象，其中的**每个元素都是唯一**的，可变集合（set）表示该集合创建后依然可以对其中元素进行增减或修改，不可变集合（frozenset）表示该集合一旦创建，元素便无法修改

```
#代码2.24 集合对象的定义和基本运算
set1 = {1,2,3,4,5} #定义可变集合对象
set2 = {3,4,5,6,7}
print(f"{3 in set1}")
print(f"{set1|set2}") #求两个集合的并集
print(f"{set1&set2 }") #求两个集合的交集
print(f"{set1-set2}") #求两个集合的差集
print(f"{set2-set1}")
set2 = set1
print(f"{set1}, {set2}")
```

# 组合数据类型——集合

- 集合是一种非序列容器对象，其中的**每个元素都是唯一**的，可变集合（set）表示该集合创建后依然可以对其中元素进行增减或修改，不可变集合（frozenset）表示该集合一旦创建，元素便无法修改

```
#判断集合set1是否为集合set2的真子集
print(f"{set1<set2}")
#判断集合set1是否为集合set2的子集
print(f"{set1<=set2 }")
set3 = frozenset({1,2,3}) #创建不可变集合
print(f"{set3}")
#不可变集合是无法对元素进行修改的，所以程序报错
set3.add(4)
```

# 集合的常用方法

```
#代码2.25 集合对象的常用方法
set1 = {1,2,3,4,5}  #定义可变集合对象
set1.add(6) #向可变集合中添加元素
print(f"After add: {set1}")

#从可变集合中删除元素，如果该元素不存在就报错
set1.remove(6)

#从可变集合中删除元素，如果该元素不存在，什么也不做
set1.discard(6)

set2 = {3,4,5,6,7}
```

# 集合的常用方法

```
#求两个集合的交集
print(f"{set1.intersection(set2)}")
#求两个集合的差集
print(f"{set1.difference(set2)}")
#用参数中的对象更新可变集合
set1.update(set2)
#判断set2是否是set1的子集
print(f"{set2.issubset(set1)}")
#判断set1是否是set2的父集
print(f"{set1.issuperset(set2)}")
```

# Python函数定义和调用

- Python函数与C语言函数基本相似，定义的关键字是**def**
- Python函数的**形式参数可以设置默认值**

```
#代码2.26 从键盘上输入一个正整数，判断其是否为素数
def isPrime(n):
    for i in range(2,n):
        if n%i==0:
            return False
    return True
num = int(input(“请输入一个大于2的正整数："))
print(f"{num}是素数" if isPrime(num) else f"{num}不是素数")
```

# Python函数定义和调用

- Python函数与C语言函数基本相似，定义的关键字是**def**
- Python函数的**形式参数可以设置默认值**

```
#代码2.27 参数的默认值
def fun(a=10, b=20, c=30):
    return f"{a} + {b} + {c} = {a+b+c}"
print(fun(15, 25, 35))  #没有使用参数默认值
print(fun(15, 20))  #参数c使用了默认值30
print(fun(b=15))  #参数a和参数c使用了默认值
print(fun())   #所有参数都采用默认值
```

# 引用传递和值传递

- Python中的变量是没有类型的，我们可以把它看做一个(*void)类型的指针，变量是可以指向任何对象的，而对象才是有类型的。
- Python中的对象有不可变对象（number，string，tuple等）和可变对象之分（list，dict等）。

1. **不可变对象**作为函数参数，相当于C语言的**值传递**。
2. **可变对象**作为函数参数，相当于C语言的**引用传递**。（因列表是可变数据类型，作为参数传递实则是传递对列表的引用，修改更新列表值后依旧引用不变）

- Python函数可以return多个值

# 引用传递和值传递

**#值传递（传递的是不可变对象）示例**

```
def modify_value(my_value):
    my_value = my_value + 1

my_integer = 5
print("Original Integer:", my_integer)

modify_value(my_integer)
print("Modified Integer:", my_integer)
```

**引用传递（传递的是对象的引用）示例**

```
def modify_list(my_list):
    my_list.append(4)

my_list = [1, 2, 3]
print("Original List:", my_list)

modify_list(my_list)
print("Modified List:", my_list)
```

```
Original Integer: 5
Modified Integer: 5
```

```
Original List: [1, 2, 3]
Modified List: [1, 2, 3, 4]
```

# 匿名函数与lambda关键字

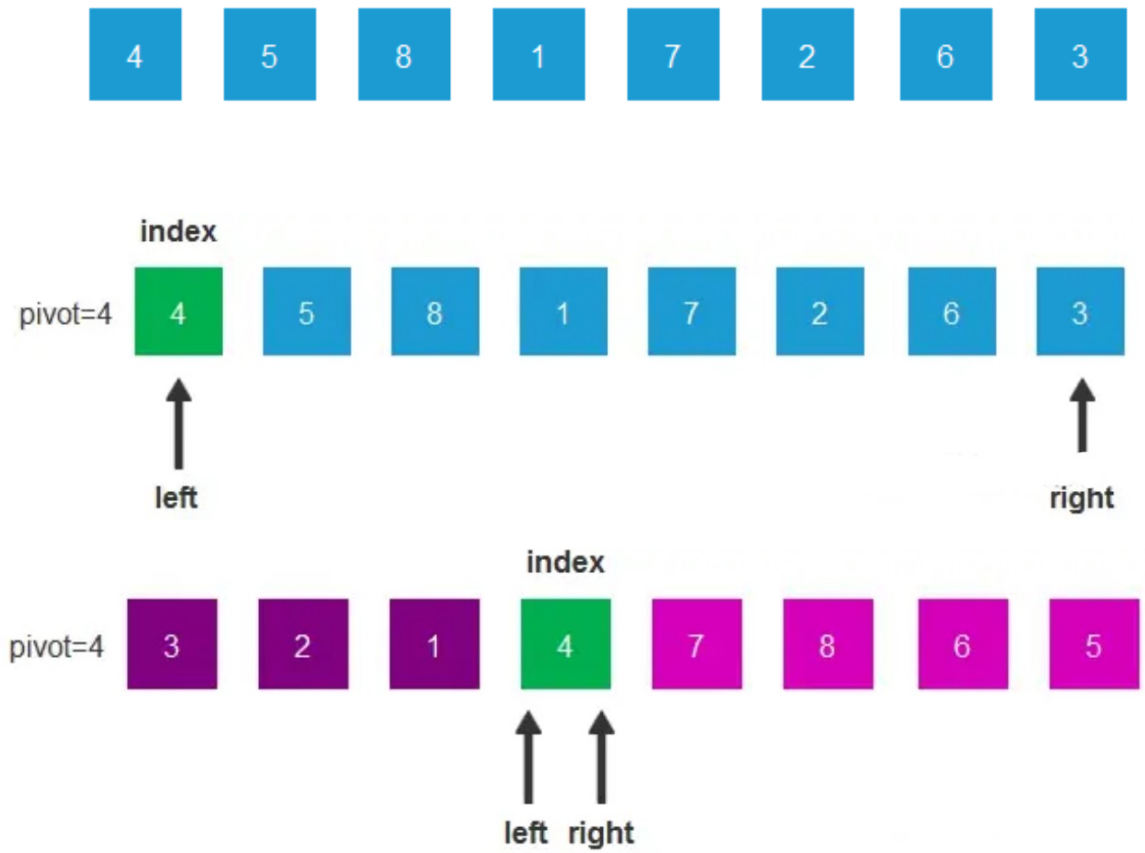


- **lambda**表达式的功能是创建一个非常短小的函数应用，将参数和返回值的关系进行约定

```
g = lambda x: x+1
g(5)
#代码2.28 匿名函数lambda表达式的应用
ages = {"Tom":20, "Jack":19, "Rose":18, "Mary":21}
#默认的排序方式是按照键的从小到大顺序
print(f"{sorted(ages)}")
#在sorted函数中，可以通过指定key参数对应的函数来改变排序的依据，以lambda表达式的返回值，即年龄进行排序
print(f"{sorted(ages, key=lambda x: ages[x]) }")
```

# 一起读代码

```python
"""
A pure Python implementation of the quick sort algorithm

For doctests run following command:
python3 -m doctest -v quick_sort.py

For manual testing run:
python3 quick_sort.py
"""
from __future__ import annotations


def quick_sort(collection: list) -> list:
    """A pure Python implementation of quick sort algorithm

    :param collection: a mutable collection of comparable items
    :return: the same collection ordered by ascending

    Examples:
    >>> quick_sort([0, 5, 3, 2, 2])
    [0, 2, 2, 3, 5]
    >>> quick_sort([])
    []
    >>> quick_sort([-2, 5, 0, -45])
    [-45, -2, 0, 5]
    """
    if len(collection) < 2:
        return collection
    pivot = collection.pop()  # Use the last element as the first pi
    greater: list[int] = []  # All elements greater than pivot
    lesser: list[int] = []  # All elements less than or equal to piv
    for element in collection:
        (greater if element > pivot else lesser).append(element)
    return quick_sort(lesser) + [pivot] + quick_sort(greater)


if __name__ == "__main__":
    user_input = input("Enter numbers separated by a comma:\n").strip()
    unsorted = [int(item) for item in user_input.split(",")]
    print(quick_sort(unsorted))
```




https://github.com/TheAlgorithms/Python/blob/master/sorts/quick_sort.py